Beaver Node Android integration guide

Development handoff for embedding node configuration inside another Android app. Contract baseline: Android app 0.2.0 and ESP32 firmware 0.2.0, HTTP API v1. Prepared 21 October 2026.

The Android app connects directly to the ESP32 access point. The ESP32 is the backend for all APIs in this guide. No cloud login, Raspberry Pi API, MQTT connection or internet access is involved in node configuration.

```
Host Android app
  -> selected Beaver Wi-Fi Network
  -> HTTP 192.168.4.1:80 /api/v1/*
  -> ESP32 settings, RTC, live sample and flash storage

STM32 -> UART -> ESP32 -> E52 mesh -> Pi gateway -> MQTT
                     (separate data delivery path)
```

Section in host app	Existing functions
Connection	Request Wi-Fi connection, manual fallback, verify device identity.
Live	Eight frequencies and NTC resistances; fault flags, age, battery and PCB temperature.
Device	Logging interval, node ID, gateway target and RTC setting.
Radio	Role, PAN, channel, rate, power, timeouts and delivery enable.
Logs	Validated CSV download, local files, sharing, recent 100 rows and frequency plot.
Health	RTC/UART/storage/radio diagnostics, delivery cursor, AP shutdown.

Scope and identity

device_id identifies the physical ESP32. node_id is the configurable logical radio address. target is the gateway-connected radio address. Do not replace these with the Pi gateway ID or MQTT Thing name. The node API still returns the ESP32 identity even though later gateway MQTT payloads use gateway identity.

All examples are illustrative, not captured live responses. The guide describes the inspected code. Confirm the firmware installed on each field device before depending on optional 0.2.0 fields.

Connection and Android network handling

Property	Current implementation
Wi-Fi	SSID Beaver; password 12345678; WPA2-PSK; channel 1; maximum 2 clients.
Address	http://192.168.4.1 on port 80. Fixed base /api/v1/.
Enable AP	INPUT_1 GPIO34 active LOW with external pull-up. LED_1 GPIO27 active HIGH. Firmware debounces over three 20 ms polls; this is not the Pi 2-second button workflow.
Boot and shutdown	AP starts off. Physical button enables it; POST ap/disable turns it off. No AP auto-expiry or credentials-setting API.
Node Wi-Fi mode	AP only. There is no node STA/router credential workflow in this firmware.

Automatic connection sequence

1. Ask the operator to enable the node AP. Request the runtime Wi-Fi permission. Construct WifiNetworkSpecifier with SSID Beaver and WPA2 passphrase 12345678. Request a TRANSPORT_WIFI Network and remove NET_CAPABILITY_INTERNET.

2. Android displays its connection consent dialog. onAvailable provides a Network object. Keep that object and open each HTTP connection through Network.openConnection. Do not use the cellular/default network for node requests.

3. GET status, validate device_id and an eight-element channels array, then GET config. Show the device identity before editing. An associated Wi-Fi network alone is not proof of a working Beaver API.

4. On network loss, clear the selected Network and live state, stop polling, and show disconnected. Keep completed local downloads accessible. Release callbacks when the feature is closed or its owner is destroyed.

Manual fallback

Open Android Wi-Fi settings, join Beaver, accept the no-internet connection, return to the app and select Use Wi-Fi. The existing implementation selects the first available Wi-Fi Network and verifies the API. A host app should handle multiple available Wi-Fi networks explicitly rather than assuming the first is correct.

Multiple nodes use the same SSID. Enable one AP at a time when practical, and verify device_id on every new connection. Do not silently reuse edited settings when switching to another node.

Android project and transport contract

Current source: native Java Activity, Java 17, minSdk 29 (Android 10), compileSdk/targetSdk 36, versionName 0.2.0, versionCode 2. Build uses Gradle 8.13 and Android Gradle Plugin 8.13.2. These are the inspected project settings, not requirements to change the host app framework.

Manifest or runtime item	Existing handling
Normal permissions	INTERNET, ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE, CHANGE_NETWORK_STATE, CHANGE_WIFI_STATE.
Android 10 to 12	ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION declarations capped at SDK 32; runtime location request. Automatic connection may also need the phone Location setting.
Android 13 and later in current target	NEARBY_WIFI_DEVICES with neverForLocation; runtime request. Reassess platform requirements when increasing targetSdk beyond the inspected 36 baseline.
Cleartext	Base policy disallows HTTP; domain exception permits only 192.168.4.1. Merge this exception into the host app network-security XML.
File access	App-private storage plus system document picker; no broad storage permission required. Read-only provider with temporary URI grants for sharing.

Request behaviour	Value
Connect / read timeout	7000 ms / 15000 ms. Read timeout is an inactivity bound, not a whole-download deadline.
Redirects	Disabled. Existing client accepts only HTTP 200 as success.
Headers	Accept: application/json, or text/csv for logs. JSON writes use Content-Type: application/json and fixed byte length.
Limits	Firmware JSON body: 1 to 1024 bytes. App JSON response: 32768 bytes; error text: 2048 bytes.
Concurrency	Single background executor and busy guard in the app. Firmware has 3 open HTTP sockets and one CSV export at a time.

Use UTF-8. JSON keys are case-sensitive. Send numeric integers as numbers and radio_enabled as a JSON boolean. Firmware rejects unknown or duplicate setting fields, arrays, nested request objects and malformed bodies. Do not echo an entire GET config response into PUT config.

This local API has no token, login or per-user authorization. Host-app roles can hide features but do not prevent another AP client from calling the API. There are no implemented CORS/OPTIONS handlers for browser-hosted clients; use a native networking bridge for a web-based host app.

API inventory and common errors

Method and path	Success and purpose
GET /api/v1/status	200 JSON: latest RAM sample, health and storage bounds.
GET /api/v1/config	200 JSON: persisted configuration plus read-only radio policy.
PUT /api/v1/config	200 JSON: full resulting configuration. Partial updates supported.
PUT /api/v1/time	200 body {"ok":true}: RTC updated.
POST /api/v1/ap/disable	200 JSON: shutdown requested with disable_after_ms 1000.
GET /api/v1/logs.csv	200 streamed CSV; optional inclusive from/to sequence query.
GET /	200 HTML: basic API links; not the app configurator.

Error envelope is not standard JSON

Most handler errors are sent with the ESP-IDF HTTP error helper as text. Do not expect {"error":...} for all failures. Preserve HTTP status and a bounded text body. A connection close or timeout may occur instead of an HTTP response. Do not show credentials in diagnostics.

Case	Meaning and app action
400 Bad Request	Malformed/invalid input or export range. Display details, correct fields or refresh storage bounds. Do not loop retries.
500 Internal Server Error	Allocation, RTC, persistence or export-start failure. Preserve message; refresh relevant state before retrying.
503 Service Unavailable	CSV export busy or AP shutting down. Wait for other export to finish or reconnect before retrying.
404 or unsupported method	Wrong API path, method, device or firmware. Treat as compatibility error; do not assume gateway endpoints exist.
Timeout / disconnect	For GET, reconnect and retry deliberately. For PUT, outcome is unknown: GET/read back before retry. For AP disable, disconnect can be expected.
Malformed 200 response	Protocol/compatibility error. Never declare success solely from the status code; validate body or CSV completion.

The JSON helper sends application/json and Cache-Control: no-store for status/config. RTC success currently does not explicitly set application/json, despite returning JSON text. Parse its successful body rather than requiring that response MIME type. AP-disable success explicitly sets application/json.

Status response and live readings

GET /api/v1/status has no body. The existing Live screen polls every 5 seconds while foreground and idle. It does not continuously poll editing screens. Latest values come from RAM, not from the last flash log.


```
{
  "device_id": "b0a7327aea29",
  "firmware_version": "0.2.0",
  "ap_on": true,
  "radio_ready": true,
  "radio_error": 0,
  "frames": 120,
  "bad_frames": 0,
  "uart_overflows": 0,
  "missed_logs": 0,
  "radio_retries": 0,
  "time_valid": true,
  "utc": 1792540800,
  "sample_age_ms": 4342,
  "stale": false,
  "BatteryPercentage": 99,
  "PcbTemperature": 31.0,
  "channels": [
    {"frequency_hz1": 2201, "resistance_ohm1": 2010, "status1": 0},
    {"frequency_hz2": 2202, "resistance_ohm2": 2020, "status2": 0},
    {"frequency_hz3": 2203, "resistance_ohm3": 2030, "status3": 0},
    {"frequency_hz4": 2204, "resistance_ohm4": 2040, "status4": 0},
    {"frequency_hz5": 2205, "resistance_ohm5": 2050, "status5": 0},
    {"frequency_hz6": 2206, "resistance_ohm6": 2060, "status6": 0},
    {"frequency_hz7": 2207, "resistance_ohm7": 2070, "status7": 0},
    {"frequency_hz8": 2208, "resistance_ohm8": 2080, "status8": 0}
  ],
  "stream_id": "357476119588692749",
  "first_sequence": "1",
  "last_sequence": "72",
  "delivery_cursor": "70",
  "record_capacity": 103168,
  "storage_fault": false
}
```


channels[0] contains frequency_hz1, resistance_ohm1 and status1; channels[7] contains the corresponding keys ending in 8. Do not look for generic frequency_hz/status keys. Frequency and resistance are integer Hz and ohms. PCB temperature is Celsius; it is not an NTC channel temperature.

stream_id, first_sequence, last_sequence and delivery_cursor are decimal strings representing unsigned 64-bit values. Keep them as strings or BigInteger/BigInt. JavaScript Number and signed Java long cannot represent the full contract safely. record_capacity in this example is illustrative; read it from the device.

Status field meanings and quality flags

Fields	Meaning
device_id / firmware_version	12 lowercase hex characters for physical ESP32 identity. firmware_version is available in 0.2.0; tolerate absence on older firmware.
time_valid / utc	RTC validity and Unix UTC seconds. utc is 0 when invalid; never display it as an actual 1970 measurement.
sample_age_ms / stale	Age since accepted STM32 sample; -1 if none. Stale at age >= 60000 ms or no sample.
BatteryPercentage / PcbTemperature	Number or JSON null. Missing fields on older firmware are unavailable. Battery 0 is valid; temperature 0 is valid. Null also occurs when stale.
frames / bad_frames / uart_overflows	Accepted frames, rejected input and UART overflow diagnostics. Treat as firmware runtime counters, not stored-log totals.
missed_logs / radio_retries	Logging failure/skipped scheduling diagnostics and radio retry counter; not a count of outstanding records.
first_sequence / last_sequence	Retained sequence bounds, 0 when empty. Holes can exist; last-first+1 is not an exact retained-record count.
delivery_cursor	Durably acknowledged radio delivery position for the current stream/target/PAN context. Not MQTT cloud confirmation.
record_capacity / storage_fault	Maximum storage capacity in records, and storage failure indicator. Capacity is not occupancy.
ap_on / radio_ready / radio_error	AP activity, E52 configuration readiness and radio diagnostic code. Readiness does not prove successful gateway reception.

Per-channel status is a bitmask

Bit value	Meaning
0	OK only when no bits are set.
1	Frequency fault.
2	NTC resistance fault.
4	Stale data.
8	No sample received.

Combine flags with bitwise tests. Status 7 = frequency fault + NTC fault + stale; 15 additionally means no sample. Unknown bits should be displayed as unknown flags. Stale channel numbers may still contain older readings: do not treat them as fresh because they look plausible.

Frequency 0 represents faulty/disconnected input; resistance -255 represents failed NTC measurement. This contract cannot distinguish NTC open circuit from short circuit. Do not label them separately without a future firmware change.

Configuration fields and defaults

GET /api/v1/config returns the full object below. These are firmware 0.2.0 fresh-device defaults, not the settings of a commissioned node. Persisted settings take precedence. In particular PAN 1 and target 2 are not your previously tested gateway PAN 4660 and address 1.

```
{
  "revision": 1,
  "interval_s": 900,
  "node_id": 1,
  "target": 2,
  "panid": 1,
  "role": 0,
  "channel": 18,
  "rate": 0,
  "power": 22,
  "route_ms": 30000,
  "link_ack_ms": 15000,
  "app_ack_ms": 90000,
  "radio_enabled": false,
  "communication_mode": "broadcast",
  "security_enabled": true,
  "encryption_key_source": "module_existing"
}
```

Writable field	Type and inclusive range
interval_s	Integer 300 to 604800 seconds. Default 900.
node_id / target	Integer node_id 0 to 65534; target 0 to 32767.
panid / role	Integer panid 0 to 65534; role 0 router or 1 terminal.
channel / rate	Integer channel 0 to 79; rate 0, 1 or 2.
power	Integer -9 to 22 dBm.
route_ms / link_ack_ms	Integer 1000 to 65535 milliseconds each.
app_ack_ms	Integer 5000 to 300000 milliseconds.
radio_enabled	JSON boolean true or false, not string or 0/1.

Read-only fields: revision, communication_mode, security_enabled, encryption_key_source. They must not appear in PUT requests. Unknown fields are rejected. Internal configuration version is not an exposed API field.

Saving settings and handling concurrent edits

Cross-field validation

Router role 0 requires node_id 0..32767. Terminal role 1 requires node_id 32768..65534. node_id must differ from target. app_ack_ms must be at least route_ms + link_ack_ms + 5000. When changing role, send the matching node ID in the same request.

```
PUT /api/v1/config
Content-Type: application/json

{"interval_s":300,"node_id":3,"target":1}

HTTP 200 -> full resulting config object
```

Only the supplied writable fields are changed. Firmware validates the merged configuration, commits it to NVS and then updates the in-memory config. A changed configuration increments revision. An unchanged patch, including an empty object, returns the existing config without increasing revision.

Existing app save flow

1. Load config before showing fields. Keep a baseline. Build a patch only for user-edited values. Validate input types/ranges and cross-field rules; show a confirmation of the patch.

2. Immediately before PUT, GET config again and validate the intended patch merged into the fresh result. Send only the patch, not the full merged object. Replace the UI baseline with the returned full config.

3. Changes to radio parameters apply asynchronously. Show Settings saved separately from Radio ready. Monitor status and delivery_cursor to distinguish module configuration from end-to-end delivery.

There is no ETag, If-Match or revision precondition. The fresh GET reduces stale-edit risk but does not make the following PUT atomic against another client. For a host app, detect baseline changes and ask the operator to resolve conflicting edits; do not present this as firmware-enforced locking.

Exact handler error text

Response	Body / trigger
400	Invalid request: invalid JSON/body, missing body or non-object input.
400	Invalid/duplicate field, address-role mismatch, or app_ack_ms below route_ms + link_ack_ms + 5000
500	Persistence failure may retain the generic text Invalid request. Do not infer that every instance of this text is a validation error; inspect HTTP status.
No response	Connection/receive failure may interrupt the request. Read config after reconnecting before deciding to repeat the write.

A changed interval restarts the logger countdown. Logging is interval-based, not aligned to quarter-hour boundaries. Changing RTC time does not restart that monotonic schedule. Raw channel values are retained; there is no configurable spike-filter feature.

Radio configuration and delivery semantics

Setting	Interpretation
role	0 router; 1 terminal. The node API accepts both; verify terminal support against the gateway implementation before deploying it.
rate	0 = 62.5K; 1 = 21.825K; 2 = 7K.
channel	Index 0..79. Existing app displays 850.125 + channel MHz for E52-900. Range acceptance is not regional RF approval.
communication_mode	broadcast in firmware 0.2.0; display-only. There is no mode selector API.
security_enabled	true; display-only. The firmware enables E52 security.
encryption_key_source	module_existing. Neither the key nor a key-setting field is exposed. Matching keys must already be provisioned in radios.
radio_enabled	Controls delivery processing. false results in radio_ready false and radio_error 0; it is not an AP control.

```
PUT /api/v1/config
Content-Type: application/json

{"node_id":3,"target":1,"panid":4660,"role":0,
 "channel":15,"rate":0,"power":22,"radio_enabled":true}
```

This example matches the previously working gateway profile: address 1, PAN 4660, channel 15, 62.5K and 22 dBm. It is not a universal default. Read the actual gateway configuration and use unique node addresses. The node default channel 18 differs from that tested gateway channel 15.

radio_error	Meaning
0	No current error; check radio_enabled and radio_ready too.
1	E52 configuration failed.
2	Stored record read failed.
3	Delivery cursor persistence failed.
4	Gateway ACK timeout / unsuccessful send path. radio_ready may remain true.

The delivery cursor is scoped to stream, target and PAN. Changing target or PAN can invalidate the current cursor and replay retained history to that destination. Warn the operator. Downloading CSV does not advance this cursor. Node radio delivery and Pi MQTT delivery are separate mechanisms; the later gateway rule about not replaying uploaded MQTT records does not change this node behaviour.

A successful PUT means settings were saved, not that E52 accepted them. A growing delivery cursor is evidence of acknowledged radio progress; it does not prove AWS storage. There is no HTTP endpoint for radio ping, per-packet ACK history, RSSI, hop count, topology or route-table editing.

RTC synchronization and AP shutdown

PUT /api/v1/time

```
PUT /api/v1/time
Content-Type: application/json

{"utc":1792540800}

HTTP 200
{"ok":true}
```

The request must contain exactly one field named utc, an integer Unix timestamp in seconds from 1704067200 through 4102444799 inclusive (2024 through 2099 UTC). Milliseconds, numeric strings, fractions, extra keys and null are invalid.

For phone synchronization, generate epoch seconds at confirmation time. Do not add the local time-zone offset: epoch seconds already represent UTC. Display times in the phone zone separately. For manually entered local time, convert using the selected zone; the existing app rejects ambiguous and nonexistent daylight-saving times.

After success, GET status and verify time_valid and utc against the phone clock. Use a small tolerance for network/processing delay. Do not treat a successful request as proof of backup-power retention; test that on hardware.

Failure	Response
Invalid body or timestamp	400: utc must be integer Unix seconds, 2024..2099
RTC write failure	500: ESP-IDF error name returned by esp_err_to_name.
Connection lost	Unknown outcome. Reconnect and read utc before retrying.

POST /api/v1/ap/disable

```
POST /api/v1/ap/disable
(no body required)

HTTP 200, application/json
{"ok":true,"disable_after_ms":1000}
```

Confirm with the operator first. The firmware sends the response, queues shutdown, waits about one second, cancels any active export and waits for its worker to finish before stopping HTTP and Wi-Fi. Actual disconnection can therefore take longer than 1000 ms.

Stop app polling and release the requested Wi-Fi Network after acknowledgment. Treat subsequent network loss as expected. The current app clears live/config state; saved phone files remain. The physical GPIO34 button is required to enable AP again; there is no remote AP-enable route while Wi-Fi is off.

AP shutdown does not save unsent form edits. Save and verify each configuration first. AP shutdown does not erase logs or disable the independently running logger/radio tasks.

CSV endpoint and record format

```
GET /api/v1/logs.csv
GET /api/v1/logs.csv?from=1&to=72

HTTP 200
Content-Type: text/csv
Content-Disposition: attachment; filename=beaver-logs.csv
```

The query range is inclusive. With no query, the firmware snapshots current bounds. If a query is supplied, both from and to are required as decimal uint64 strings: from > 0, from <= to and to <= current last_sequence. A start before current first_sequence is rejected. The client should validate both bounds against a fresh status response.

Current export has 34 columns in exactly this order. The legacy format has the same first 32 columns, without the two auxiliary fields. Actual header is one line; it is wrapped below only for readability.

```
kind,sequence,stream,utc,node_id,age_ms,time_flags,
F1,F2,F3,F4,F5,F6,F7,F8,
R1,R2,R3,R4,R5,R6,R7,R8,
S1,S2,S3,S4,S5,S6,S7,S8,
config_revision,BatteryPercentage,PcbTemperature
```

Column group	Meaning
kind	DATA for records. ERROR lines are control/error records, not measurements.
sequence / stream	Decimal uint64 values. Use stream plus sequence to identify a stored record.
utc / time_flags	Unix seconds; time_flags bit 1 means invalid RTC timestamp. Invalid utc is 0.
node_id / config_revision	Logical node ID and configuration revision captured when that log was created.
age_ms	Age of the source sample when logged. No sample uses 4294967295, unlike live status which uses -1.
F1..F8 / R1..R8	Integer Hz / integer ohms. Fault and stale values must be interpreted with S flags.
S1..S8	Same channel quality bitmask as live status.
BatteryPercentage	Integer 0..100 or blank when unavailable.
PcbTemperature	Celsius decimal with two digits in export, or blank when unavailable.

Blank auxiliary CSV cells must remain unavailable, never zero. Records from the older flash format can have blank auxiliaries even in a 34-column export. Temperature is PCB temperature only; NTC R fields remain resistance.

The endpoint does not pin flash pages. Ring-buffer overwrite can race an export. A normal HTTP 200 start is therefore not proof that a complete file was delivered.

Reliable downloads and export errors

Completion algorithm used by the app

1. GET status; capture stream_id, first_sequence and last_sequence. For full export, explicitly request those nonzero bounds. For an empty store (last_sequence 0), use logs.csv without a query and accept a header-only file.

2. Stream into a temporary .partial file. Validate the exact 32- or 34-column header; each row must be DATA with the matching column count. Reject ERROR rows. Preserve trailing blank cells when splitting CSV.

3. Require each row to have the captured stream, a strictly increasing sequence within the requested range, and numeric values. Duplicate/descending records fail. The existing parser checks battery 0..100 and finite PCB temperature -327.67..327.67 when present. It checks other data columns as integer values, not every physical range.

4. At EOF require the final sequence to equal the requested to. Holes within the range are allowed; row count need not equal to-from+1. A custom ending sequence must correspond to a retained valid record. If it ends on a hole, the current app rejects the export as incomplete.

5. Only after validation succeeds, rename to beaver-<device_id>-<phone_epoch_ms>.csv in app-private files. Delete the partial on failure. Keep only the newest 100 rows in memory for the preview while counting all DATA rows.

Status or stream content	Handling
400: Use ?from=<sequence>&to=<sequence>, within available range	Invalid/oversized query or out-of-range end. Refresh bounds and correct input.
400: Start sequence no longer retained	Oldest requested data was overwritten. Inform operator; obtain new bounds.
503: Export unavailable or AP shutting down	Another export or shutdown is active. Avoid parallel exports.
500: Cannot start export	Export worker could not start. Some async allocation failures can instead close the connection.
ERROR,export_cancelled	Shutdown interrupted export. Discard the partial file.
ERROR,range_overwritten_during_export	Ring advanced beyond needed data during export. Discard partial; refresh bounds.
ERROR,storage_read_failed	Storage read error. Discard partial; inspect storage health.
EOF before requested final sequence	Incomplete export even if HTTP was 200. Do not save as complete.

There is no resume token or date-range filter API. Restart a failed snapshot/range download after refreshing bounds. The export is not a background service in the existing app. Keep the feature open; process termination may require retry. Deleting a phone copy never deletes device records.

Embedding the feature in another app

Recommended module boundaries

Module	Responsibility and reusable source
Connection controller	Own Android Network requests/callbacks and lifecycle. Extract connection logic from MainActivity.
Node API client	Reuse DeviceApi request behaviour, selected-Network routing, timeouts and bounded error parsing.
Models and validation	Reuse Protocol field constraints, flags and uint64 handling. Separate response-only fields from writable patch DTOs.
Feature state	Use host app state-management pattern for connection, loading, editing, errors and selected node. Avoid coupling device identity to screen navigation.
Log repository	Reuse CsvLog stream validation. Own partial/final files, metadata and 100-row preview.
Screens and sharing	Recreate Live/Device/Radio/Logs/Health in host design. Adapt LogProvider authority and document-picker integration.

For Flutter, React Native or a WebView host, put Wi-Fi selection and Network-bound HTTP in an Android native bridge. A generic fetch/Dart HTTP call is not automatically bound to the local-only Network returned by WifiNetworkSpecifier. Return parsed objects and file handles/events to the UI.

```
// Transport pattern already used by DeviceApi
HttpURLConnection c = (HttpURLConnection) selectedNetwork
    .openConnection(new URL(
        "http://192.168.4.1/api/v1/status"));
c.setRequestMethod("GET");
c.setConnectTimeout(7000);
c.setReadTimeout(15000);
c.setInstanceFollowRedirects(false);
```

Suggested connection states: disconnected, permission required, connecting, verifying device, ready, unavailable, shutting down. Keep a separate operation state for saving, syncing and exporting. A radio error must not be presented as a Wi-Fi connection failure.

Serialize device operations; suspend Live polling during writes/downloads. Never perform network or CSV work on the UI thread. Clear stale screen values on disconnect or mark them explicitly as last read. Handle process recreation rather than relying only on orientation handling.

The existing ContentProvider authority is com.cyamsys.beaver.logs and is hard-coded in sharing logic. Change both manifest and URI construction for the host applicationId to avoid installation conflicts. Grant temporary read permission; do not expose arbitrary private files.

Acceptance tests and known limitations

Test	Expected result
Wi-Fi with mobile data on	API calls use Beaver Wi-Fi despite no internet. Permission denial/cancel offers manual fallback.
Node identity	Switch between two nodes with the same SSID; no old config or readings are silently reused.
Live data quality	Eight correct channel mappings; 0 vs null preserved; stale at 60 s; status 7 and 15 decoded as flags.
Settings validation	Reject strings/fractions/out-of-range values; update role and ID together; reject node equal target and short ACK timeout.
Write outcome	Successful PUT readback matches edits. Lost response triggers readback, not a blind repeated write.
RTC	Phone sync and manual local time convert correctly; invalid date/body returns 400; verify time_valid after success.
Radio	Saved != ready != delivered. Match gateway profile; observe delivery cursor. Confirm replay warning on PAN/target changes.
CSV	Accept old/new formats and empty store. Reject wrong stream, duplicates, truncation, ERROR lines and missing ending record.
Local files	Save/share complete CSV; uninstall removes app-private copy but not independently exported file. Phone delete leaves flash unchanged.
Lifecycle	Background, resume, rotate, lose Wi-Fi, stop app during export. No partial file is offered as complete.
AP disable	Warn about unsaved edits, receive acknowledgment or expected disconnect, re-enable by physical button.

Limitations to preserve or improve explicitly

No app-account authentication in the current node backend; no HTTPS, API key, OTA, AP credential update, log erase, NTC beta conversion, exact live log count, node STA mode or remote AP enable. None should be represented as implemented features.

Read-only config/security fields and firmware_version are optional on older firmware. The Android Health footer still contains the old 0.1.0 text even though Gradle identifies the app as 0.2.0; use build metadata, not that footer, when labelling the integrated release.

CSV download validation is stronger than reopening saved files: loadLog performs lighter parsing. If the host app will import externally supplied CSV files, run full validation before viewing them. Keep downloaded previews bounded and do not interpolate faulty readings in charts.

Source map and handoff checklist

The code can be handed over separately later. This document defines the existing behaviour so the receiving developer can integrate it without inventing endpoints or coupling it to the gateway configurator.

Source under outputs	Relevant contract
beaver-android/app/src/main/java/com/cyamsys/beaver/DeviceApi.java	Network-bound HTTP, request/response limits and errors.
beaver-android/.../MainActivity.java	Connection callbacks, screen flows, save/readback, time, exports and AP shutdown.
beaver-android/.../Protocol.java	Settings ranges, combined validation, uint64 and quality flags.
beaver-android/.../CsvLog.java and LogProvider.java	Download validation, preview and file sharing.
beaver-android/app/src/main/AndroidManifest.xml and res/xml/network_security_config.xml	Permissions, provider and host-scoped HTTP exception.
beaver-node-v0.2.0/main/web.c	HTTP routes, response shapes, error strings and AP behaviour.
beaver-node-v0.2.0/main/settings.c	Defaults, persistent updates, read-only policy and validation.
beaver-node-v0.2.0/main/storage.c and radio.c	Cursor scoping, async configuration and delivery status.
beaver-node-v0.2.0/components/beaver_core/core.c	Stored-record quality flags, age and auxiliary-field handling.

Repository references: https://github.com/Kuldeep-Cyamsys/beaver-node-app for Android and https://github.com/Kuldeep-Cyamsys/esp32_VW_dev for firmware. The API contract here was checked against local source snapshots; the remote branch was not used to infer installed firmware.

The separate C:/Users/cyamsys/hello_world checkout still has older config defaults and lacks the 0.2.0 policy/version response additions. Do not use its defaults as the 0.2.0 contract. Exact relevant file SHA256 fingerprints are included in source-manifest.json beside this guide.

For the later code handoff include Java sources, manifest/resources, Gradle wrapper/build settings, unit tests, this contract and a test APK. Exclude build caches, local.properties paths and signing credentials. No changes to firmware or Android production code were made for this documentation task.

Before acceptance: choose the host applicationId and permission owner, implement the native connection bridge if required, map all six device API operations, preserve the settings/readback and CSV integrity rules, then run the hardware acceptance matrix on representative phones and the intended firmware.